

让 Claude 普通窗口和 Claude Code 实时对话

怎么在你平时用的普通 Chat 窗口里，也能和 Claude Code 对话？让 CC 帮你看电脑文件，在电脑上放歌？

✨ 上一篇教程 (1.1) 聊了记忆系统的运作原理。这一篇来说另一个大家很关心的话题：如何通过自己的域名，在普通窗口也能和 Claude Code 对话。

一、为什么不用官方的 Dispatch

Claude.ai 自带了一个让 Chat 连接 CC 的功能，叫 Dispatch。但它目前有几个痛点：

1. **经常断开连接**，重连要重新加载大量工具，非常慢。
2. **单独的窗口**。不是你平时聊天的窗口。

所以我想做的是：在自己平时用的普通 Chat 窗口里联系 CC。不用 Dispatch，不断连，我的窗口、我的规则、我的记忆全都在。

✨ Dispatch 是官方的“开箱即用”方案，但目前还不够稳定。自建的方案配好之后更稳定、更灵活。

二、你需要准备什么

准备项	说明
一台电脑或云服务器	跑 MCP Server 和 CC。Mac/Windows/Linux 都行
一个 Claude 账号	Pro 或 Max 订阅（Custom Connector 和 CC 都需要）
电脑上装好 Claude Code	命令行里能跑 claude 就行
Python 3.10+	写 MCP Server 用。不用你写，让 CC 写
一个域名 + Cloudflare 账号	详见 1.1 第五章

这台电脑不一定是你面前的笔记本。也可以是云服务器（VPS）——服务器 24 小时在线，手机上打开 Chat 就能指挥 CC。本地电脑方案适合在家用，云服务器方案适合随时随地用。

✨ 域名、Tunnel、Custom Connector 这三个概念在 1.1 第五章有详细解释，还没看过的可以先回去翻一下。

三、它是怎么工作的

Chat 通过 MCP 工具下发指令 → 电脑上的脚本调用 CC 执行 → 结果存本地 → Chat 再来取。

Chat 和 CC 之间没有直接连接。MCP Server 是中间人，数据库是传话的。
这就是为什么不会断连——每次都是独立的请求，不需要维持连接。

电脑端的 MCP Server 提供三个工具：

工具名	做什么
cc_execute	接收任务，启动 CC 执行，返回 task_id。可传 session_id 继续上次会话
cc_check	用 task_id 查状态和结果。完成后返回结果 + session_id
cc_tasks	列出最近的任务

核心流程：cc_execute 派任务 → 拿到 task_id → cc_check 查结果（拿到 session_id）→ 需要继续时把 session_id 传给下一次 cc_execute。

如果不传 session_id，每次都是全新的 CC；传了，CC 就只记得上次做了什么。

举个例子：

1. 你：“帮我用 CC 看看项目结构”
2. Chat → cc_execute → 拿到 task_id
3. 等十几秒，Chat → cc_check → 拿到结果 + session_id

4. 你：“再帮我改一下那个 bug”

5. Chat → cc_execute(修复 bug, session_id) → CC 是同一个窗口，直接开干

✨ *cc_execute 是异步的——派完任务立刻返回，不会卡住 Chat。CC 启动需要 10-20 秒，复杂任务可能更久。如果 Chat 这轮超时了也没关系，任务在后台跑着不会中断，发下一条消息 Chat 就会自动继续查。*

✨ *技术细节不用你操心。把这篇教程发给你的 CC，告诉它“按这个思路帮我写一个 MCP 服务器”，它就知道该怎么做。或者你也可以下方群里领完整安装脚本。*

四、谁做什么

电脑端（让 CC 帮你做）

写 MCP Server。提供 cc_execute、cc_check、cc_tasks 三个工具。把这篇教程的思路告诉 CC，让它写。

配 Cloudflare Tunnel。把服务器暴露到域名上，Chat 才能找到。

保持运行。MCP Server 和 Tunnel 需要一直开着。可以设置开机自动启动。

Chat 端（你自己在 claude.ai 里做）

添加 **Connector**。Settings → Integrations，把域名加进去。详见 1.1 第五章。

写 **Profile**。教 claude 什么时候用这些工具、怎么用。比如在 Settings → Profile 里写：

“当我说 ‘用 CC’ ‘帮我去 CC’ ‘让 CC’ 或需要操作本地电脑时，用 cc_execute。流程：1. cc_execute 拿 task_id；2. 等 15 秒，cc_check 拿结果和 session_id；3. 继续时传 session_id。不传每次都是全新的 CC。如果需要继续之前的 CC 会话但不记得 session_id，先调 cc_tasks 查最近的任务列表。”

✨ 总结：电脑端交给 CC（写服务器、配 Tunnel），Chat 端你自己做（加 Connector、写 Profile）。代码让 CC 写，你只需理解思路。

五、在电脑上看 CC 的完整回复

cc_check 返回的是 CC 的最终结果，但看不到它执行过程中的完整对话。想看完整回复，需要在电脑上接入 CC 的会话。

Chat 远程创建的 CC 会话保存在你电脑本地。你可以用 `claude --resume <session_id>` 从你的终端接入，看到 CC 做了什么、怎么做的，还能继续在本地操作。

如果不想每次手动翻 `session_id`，我有一个脚本可以一键继续和电脑端 CC（和 Chat 说过话的那个窗口）对话。需要的可以群里拿（非必须），让 CC 给你写一个也可以。

✨ MCP Server 的终端窗口里也能看到 CC 的输出日志，不过没有 *resume* 那么直观。

六、常见问题

“电脑合盖 / 睡眠了怎么办？”

关闭自动睡眠，或用 `caffeinate` 命令防止休眠。或者直接把 CC 和 MCP Server 都部署到云服务器上，不用担心电脑关机。

“安全吗？别人知道域名能派任务吗？”

不能。MCP Server 有 OAuth 鉴权，服务器端有密钥，只有正确密钥的请求才接受。第一次连接时授权一次，之后自动鉴权。密钥存在本地，别传到 GitHub 上。

✨ 进阶：以后想让多个频道（比如 Telegram、微信）也能和 CC 协作，可以加一个“消息总线”功能。先跑通 Chat 和 CC 再说。

Tips:

我已经把这套系统开源了：

- **imprint-memory**：给 Claude Code 加持久记忆的 MCP 服务，支持存储、搜索、日志，单独能用。
- **claude-imprint**：基于 imprint-memory 的完整框架，加上 Telegram/微信渠道、定时心跳、自动化任务、仪表盘等。适合想搭一整套让 claude 变成类似小龙虾系统的人。（重型，适用 max 订阅，最好有电脑）

只想要记忆功能装 imprint-memory 就够了，想要完整体验装 claude-imprint。

GitHub 搜 Qizhan7/imprint-memory 和 Qizhan7/claude-imprint。